LeetCode Practice

7 Easy: Sorting

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

7
EASY

Patterns (in order)

1. Sorting (7)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

7 questions | tap a title to jump to it

Easy questions (7) Go >

Sorting (7) Go >

E 1. Majority Element LC 169

E 2. Contains Duplicate LC 217

E 3. Valid Anagram LC 242

E 4. Meeting Rooms LC 252

E 5. Third Maximum Number LC 414

E 6. Relative Sort Array LC 1122

E 7. Largest Unique Number LC 1133

Easy questions (7)

EASY | TOPIC 1 OF 1

Sorting

7 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Majority Element LC 169
2. Contains Duplicate LC 217
3. Valid Anagram LC 242
4. Meeting Rooms LC 252
5. Third Maximum Number LC 414
6. Relative Sort Array LC 1122
7. Largest Unique Number LC 1133

EASY | Sorting | LeetCode #169 | 1 of 7 | 1/7

Majority Element

QUESTION

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than n / 2 times. You may assume that the majority element always exists in the array.

Example 1	
Input	`nums = [3,2,3]`
Output	3

Example 2	
Input	`nums = [2,2,1,1,1,2,2]`
Output	2

CONSTRAINTS

- n == nums.length
- $1 <= n <= 5 * 10^4$
- $10^9 <= nums[i] <= 10^9$
- The input is generated such that a majority element will exist in the array.
- Follow-up: Could you solve the problem in linear time and in O(1) space?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

The majority element is the one that appears MORE THAN n/2 times.

We're guaranteed it always exists. We return the element, not its count. Right?"

"A few quick questions:

Is there always exactly one majority element? Yes — anything above n/2 means

at most one element can qualify.

Can n be 1? Yes — that single element is trivially the majority.

Can values be negative? Yes, constraints allow down to -10^9.

The follow-up asks for linear time and O(1) space — I'll aim for that."

"Let me walk the examples:

nums=[3,2,3]: 3 appears 2 times, n/2=1. 2 > 1 -> 3 is majority. ok

nums=[2,2,1,1,1,2,2]: 2 appears 4 times, n/2=3.5. 4 > 3.5 -> 2. ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for element in elements:
    count how many times it occurs by scanning
        the array
return the one that occurs more than n/2 times
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per element. Boyer-Moore voting (keep a candidate and a counter) finds the majority in O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

To get O(1) space, I'll use Boyer-Moore Voting Algorithm.

The core insight: if we cancel every occurrence of the majority element

with a different element, the majority element will always survive

because it has more than n/2 appearances — more than everything else combined.

Keep a candidate and a count:

When count == 0, adopt the current element as the new candidate.

If current element matches candidate, increment count.

If it doesn't match, decrement count (a cancellation).

The candidate at the end is guaranteed to be the majority element.

Pseudocode:

candidate, count = None, 0

```
for x in nums:
if count == 0: candidate = x
count += (1 if x == candidate else -1)
return candidate
```

Time: O(n) — single pass. Space: O(1) — two variables only.

PHASE 4 CLEAN CODE

Now I'll implement Boyer-Moore with meaningful names.

```python
class _169_MajorityElement:
    def majorityElement(self, nums: List[int]) -> int:
        candidate = None
        count     = 0

        for num in nums:
            if count == 0:
                candidate = num
            count += 1 if num == candidate else -1
```

```
        return candidate
```

PHASE 5 TEST & DEBUG

"Let me trace after sorting—what property does sorting give me, and how the scan uses it."

"Let me trace through both examples."

nums=[3,2,3]

num=3: count=0 -> candidate=3. count=1.

num=2: count=1, 2!=3 -> count=0.

num=3: count=0 -> candidate=3. count=1.

`return 3 ok`

nums=[2,2,1,1,1,2,2]

num=2: count=0 -> candidate=2. count=1.

num=2: match -> count=2.

num=1: no match -> count=1.

num=1: no match -> count=0.

num=1: count=0 -> candidate=1. count=1.

num=2: no match -> count=0.

num=2: count=0 -> candidate=2. count=1.

`return 2 ok`

Single element — nums=[7]

count=0 -> candidate=7. count=1. return 7 ok

Even when the candidate flips mid-array, the majority element reclaims it before the end.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
single pass	only two variables

Boyer-Moore is the optimal solution here and worth naming by name —

it shows algorithm knowledge beyond just 'I used two variables'.

Correctness argument worth stating: the majority element appears more than n/2 times.

Every time it gets cancelled by a different element, it loses one vote and so does

the other element. Since it has more votes than all others combined, it will always

have votes remaining when everyone else is cancelled out.

Alternative O(n) time O(1) space approach: sort and return nums[n//2].

Since majority appears > n/2 times, it must occupy the middle index.

But sorting is O(n log n) — worse than Boyer-Moore.

If the problem didn't guarantee a majority exists, Boyer-Moore would need

PHASE 6 COMPLEXITY & FOLLOW-UP

a second pass to verify the candidate actually appears > n/2 times.

Given more time I'd ask: what if we need to find all elements appearing

more than n/3 times? Boyer-Moore generalises — maintain two candidates

and two counts. That's a well-known variant worth mentioning.

EASY | Sorting | LeetCode #217 | 2 of 7 | 2/7

Contains Duplicate

QUESTION

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

Example 1

Input	`nums = [1,2,3,1]`
Output	`true`
Explanation	*The element 1 occurs at the indices 0 and 3.*

Example 2

Input	`nums = [1,2,3,4]`
Output	`false`
Explanation	*All elements are distinct.*

Example 3

Input	`nums = [1,1,1,3,3,4,3,2,4,2]`
Output	`true`

CONSTRAINTS

- $1 <= nums.length <= 10^5$
- $10^9 <= nums[i] <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if any value appears at least twice, false if every element is distinct.

We're not asked which value or how many times — just a boolean. Right?"

"A few quick questions:

Can the array be empty? Constraints say length >= 1, so no.

Can there be negative numbers? Yes, down to -10^9.

Is order relevant? No — we just care about value frequency."

"Let me walk the examples:

[1,2,3,1]: 1 appears twice -> true ok

[1,2,3,4]: all distinct -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] == nums[j]:
            return True
return False
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

n(n-1)/2 comparisons. A hash set of seen values is O(n) time and O(n) space; sorting and comparing neighbours is O(n log n) with O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

We're repeatedly asking: have I seen this value before?

A hash set answers that in O(1). Walk the array once;

`if the current number is already in the set, return True immediately.`

Otherwise add it and continue.

Time: O(n). Space: O(n).

Alternative: sort and check adjacent pairs — O(n log n), O(1) extra space.

Hash set is faster; sort is space-optimal.

PHASE 4 CLEAN CODE

```python
class _217_ContainsDuplicate:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace containsDuplicate on [1,2,3,1]."

`seen = set()`

i=0: 1 not in seen -> seen={1}

i=1: 2 not in seen -> seen={1,2}

i=2: 3 not in seen -> seen={1,2,3}

i=3: 1 in seen -> return True ok

Trace [1,2,3,4]: never finds duplicate -> return False ok

WHY set: O(1) lookup vs O(n) for list; early exit on first duplicate.

Edge: empty or single-element array -> always False (no pairs possible).

Alternative: sort then check adjacent — O(n log n) but O(1) space.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)** for the set

One-liner alternative: return len(nums) != len(set(nums)) — same complexity,

builds the full set before checking. The explicit loop returns early on first duplicate.

For adversarial inputs (duplicate at the end) they're the same; for duplicates early,

the loop wins. In an interview I'd use the explicit loop and mention the one-liner.

Given more time I'd ask: what if the array is sorted? Check adjacent pairs — O(n) time, O(1) space.

IN YOUR APP | Expose | Editorial | Community

EASY | Sorting | LeetCode #242 | 3 of 7 | 3/7

Valid Anagram

QUESTION

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1	
Input	`s = "anagram", t = "nagaram"`
Output	`true`

Example 2	
Input	`s = "rat", t = "car"`
Output	`false`

CONSTRAINTS

- 1 <= s.length, t.length <= $5 * 10^4$
- s and t consist of lowercase English letters.
- Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

t is an anagram of s if t contains exactly the same characters with the same frequencies, just in a different order. Right?

Both strings consist of only lowercase English letters — so 26 possible chars."

"Quick checks:

If lengths differ they can't be anagrams — I'll short-circuit on that.

Follow-up asks about Unicode — I'll address that in Phase 6."

"'anagram'/'nagaram': same chars, same counts -> true ok

'rat'/'car': r,a,t vs c,a,r — 't' vs 'c' mismatch -> false ok"

Can the string be empty? Are all characters ASCII or could they be Unicode?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort both strings
return they are equal
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

The sorts dominate. Counting the 26 letters (add for s, subtract for t) and checking all zeros is O(n) time and O(1) space.

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Instead of sorting, count character frequencies.

Since only lowercase letters exist, a 26-slot array suffices — O(1) space.

Increment for each char in s, decrement for each char in t.

If all slots are zero at the end, they're anagrams.

Time: O(n). Space: O(1) — fixed 26-slot array.

PHASE 4 CLEAN CODE

```python
class _242_ValidAnagram:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        freq = [0] * 26
        for c in s:
            freq[ord(c) - ord('a')] += 1
        for c in t:
            freq[ord(c) - ord('a')] -= 1
        return all(x == 0 for x in freq)
```

PHASE 5 TEST & DEBUG

"Let me trace isAnagram on s='anagram', t='nagaram'."

count array size 26 (a-z)

For each char in s: increment count[c-'a']

For each char in t: decrement count[c-'a']

s='anagram': a+3, n+1, g+1, r+1, m+1

t='nagaram': n-1, a-3, g-1, r-1, m-1

Final count: all zeros -> True ok

Trace s='rat', t='car': r+1,a+1,t+1 then c-1,a-1,r-1 -> t count != 0 -> False ok

WHY count array: O(n) time, O(1) space (fixed 26 letters).

Trace: s="anagram", t="nagaram" -> count s: {a:3,n:1,g:1,r:1,m:1} -> subtract t: all -> 0 -> True. s="rat", t="car" -> count r:1,a:1,t:1; subtract c:1,a:1,r:1 -> c=-1 -> False.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1) 26-slot array is a constant

Follow-up: Unicode characters -> the 26-slot array breaks.

Use a Counter (hash map) instead — O(n) time, O(k) space where k is distinct chars.

`return Counter(s) == Counter(t)`

Handles any alphabet. In an interview, mention this unprompted.

EASY | Sorting | LeetCode #252 | 4 of 7 | 4/7

Meeting Rooms

QUESTION

Given an array of meeting time intervals where intervals[i] = [start$_i$, end$_i$], determine if a person could attend all meetings.

Example 1

Input	`intervals = [[0,30],[5,10],[15,20]]`
Output	`false`

Example 2

Input	`intervals = [[7,10],[2,4]]`
Output	`true`

CONSTRAINTS

- 0 <= intervals.length <= 10^4
- intervals[i].length == 2
- 0 <= start$_i$ < end$_i$ <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

We have a list of meetings [start, end]. We need to check if one person

can attend ALL of them — meaning no two meetings overlap.

The problem says meetings ending at t and starting at t do NOT overlap. Right?"

"Quick questions:

Can intervals be empty? Yes — 0 meetings, trivially true.

Are start and end guaranteed start < end? Yes, constraints say 0 <= start < end."

"[[0,30],[5,10],[15,20]]: 0–30 overlaps with 5–10 -> false ok

[[7,10],[2,4]]: 2–4 ends before 7–10 starts -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (a, b) in pairs(meetings):
    if they overlap:
        return False
return True
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Compares every pair. Sort by start time; then only neighbours can overlap, so one pass is O(n log n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sort by start, check consecutive overlap — O(n log n). Walk-through:"

Sort intervals by start time.

For i in 1..n-1:

If intervals[i][0] < intervals[i-1][1]: <- new meeting starts before previous ends

`return False (overlap -> can't attend all).`

Return True.

WHY sorting by start enables single-pass check: after sorting, if any meeting

overlaps it must overlap with the immediately preceding one (since all earlier

meetings started earlier). No need to check all pairs.

Time: O(n log n). Space: O(1).

PHASE 4 CLEAN CODE

```
class _252_MeetingRooms:
    def canAttendMeetings(self, intervals: List[List[int]]
) -> bool:
        intervals.sort()
        for prev, curr in pairwise(intervals):
            if prev[1] > curr[0]:
                return False
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace canAttendMeetings([[0,30],[5,10],[15,20]])."

Sort by start time: [[0,30],[5,10],[15,20]]

Compare consecutive: intervals[1].start=5 < intervals[0].end=30 -> overlap -> return False ok

Trace [[7,10],[2,4]]: sort->[[2,4],[7,10]]. intervals[1].start=7 > intervals[0].end=4 -> no overlap -> True ok

WHY sort by start: only need to compare adjacent intervals after sorting.

Edge: 0 or 1 meeting -> always True (no conflict possible).

Note: touching intervals [0,5],[5,10] -> 5==5, not overlap for this problem (depends on spec).

Generalize: sort ensures adjacent comparison catches all overlaps; O(n log n) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log n) for sort, O(n) for the pass	O(1) extra

pairwise is Python 3.10+; fallback: for i in range(1, len(intervals)).

Given more time I'd ask: what's the minimum number of rooms needed?

PHASE 6 COMPLEXITY & FOLLOW-UP

That's Meeting Rooms II (#253) — use a min-heap of end times, O(n log n).

IN YOUR APP | Expose | Editorial | Community

EASY | Sorting | LeetCode #414 | 5 of 7 | 5/7

Third Maximum Number

QUESTION

Given an integer array nums, return the third distinct maximum number in this array. If the third maximum does not exist, return the maximum number.

Example 1

Input	`nums = [3,2,1]`
Output	`1`
Explanation	*The first distinct maximum is 3.* *The second distinct maximum is 2.* *The third distinct maximum is 1.*

Example 2

Input	`nums = [1,2]`
Output	`2`
Explanation	*The first distinct maximum is 2.* *The second distinct maximum is 1.* *The third distinct maximum does not exist, so the maximum (2) is returned instead.*

Example 3

Input	`nums = [2,2,3,1]`
Output	`1`
Explanation	*The first distinct maximum is 3.* *The second distinct maximum is 2 (both 2's are counted together since they have the same value).* *The third distinct maximum is 1.*

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 2^{31} <= nums[i] <= 2^{31} - 1
- Follow up: Can you find an O(n) solution?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We want the third DISTINCT maximum. If fewer than 3 distinct values exist,

we return the maximum. And duplicates don't count toward the ranking — so

[1,1,2] has only 2 distinct values and we'd return 2? Right?"

"A few quick questions:

Can values be negative? That matters for my choice of sentinel.

What's the size constraint on nums?"

"Let me trace: nums=[2,2,3,1]. Distinct values: {1,2,3}.

Third max = 1 ok. And nums=[1,2]: only 2 distinct -> return 2 ok.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
sort the distinct values in descending order
return the third if it exists, else the largest
```

TIME	SPACE
O(n log n)	O(n)

WHY IT IS SLOW

Sorts everything to read three values. Keep the three largest distinct values in three variables while scanning: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is sorting when we only need the top 3.

I'll maintain three variables — first, second, third — tracking the current top tier.

I use float('-inf') as the initial value so it works even with very negative integers.

For each number: skip if it already appears in our top 3;

otherwise cascade it down from first to second to third as appropriate.

Pseudocode:

```
first = second = third = -inf
for n in nums:
if n in (first, second, third): continue
if n > first: third=second; second=first; first=n
elif n > second: third=second; second=n
elif n > third: third=n
return third if third != -inf else first
```

Time: O(n) — one pass, constant comparisons. Space: O(1) — three scalars.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach.

```python
class _414_ThirdMaximumNumber:
    def thirdMax(self, nums: List[int]) -> int:
        first = second = third = float('-inf')
        for n in nums:
            if n in (first, second, third):
                continue
            if n > first:
                third = second; second = first; first = n
            elif n > second:
                third = second; second = n
            elif n > third:
                third = n
        return third if third != float('-inf') else first
```

PHASE 5 TEST & DEBUG

"Let me trace after sorting—what property does sorting give me, and how the scan uses it."

nums=[2,2,3,1]:

n=2: first=2. n=2: dup, skip. n=3: 3>first=2 — third=-inf,second=2,first=3.

n=1: 1<second=2, 1>third=-inf — third=1.

third=1 != -inf -> return 1 ok

nums=[1,2]:

n=1: first=1. n=2: 2>1 — second=1,first=2.

third still -inf -> return first=2 ok

nums=[-1,-2,-3]:

first=-1, second=-2, third=-3. third!=-inf -> return -3 ok

Float('-inf') handles negative inputs correctly

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one pass, constant-work per element	three scalar variables

Using float('-inf') as the sentinel is critical: 0 or None would fail on arrays containing very negative integers.

Follow-up: kth Largest Element (LC 215) — generalise to arbitrary k with a min-heap of size k;

heap gives O(n log k) time and O(k) space.

Follow-up: return the actual third-maximum value AND its index — track index alongside value.

EASY | Sorting | LeetCode #1122 | 6 of 7 | 6/7

Relative Sort Array

QUESTION

Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in arr2 are also in arr1.

Sort the elements of arr1 such that the relative ordering of items in arr1 are the same as in arr2. Elements that do not appear in arr2 should be placed at the end of arr1 in ascending order.

Example 1	
Input	`arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]`
Output	`[2,2,2,1,4,3,3,9,6,7,19]`

Example 2	
Input	`arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]`
Output	`[22,28,8,6,17,44]`

CONSTRAINTS

- 1 <= arr1.length, arr2.length <= 1000
- 0 <= arr1[i], arr2[i] <= 1000
- All the elements of arr2 are distinct.
- Each arr2[i] is in arr1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sort arr1 so elements appear in the order defined by arr2.

Elements in arr1 not present in arr2 go at the end, sorted ascending. Right?

arr2 elements are distinct; every arr2 element is guaranteed to exist in arr1."

"arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]

-> [2,2,2,1,4,3,3,9,6,7,19]: follow arr2 order for those nums, then 7,19 ascending ok"

Are there duplicate elements? Should the sort be stable?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for value in arr2_in_order:
    scan arr1 and move every occurrence into the
        result
then append the remaining values sorted
```

TIME	SPACE
O(n * m + k log k)	O(k)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

A scan of arr1 per arr2 value. Count the values of arr1 (array of size 1001), write each arr2 value count times, then the rest in increasing order: O(n + m + 1001).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count sort using order array — O(n + m). Walk-through:"

`order_map = {order[i]: i for i in range(len(order))}.`

Partition arr into: elements in order_map (sorted by their index in order) and remaining.

Result = elements_in_order + elements_not_in_order.

HOW: bucket sort — buckets[order_map[x]].append(x) for x in arr if x in order_map.

`remaining = [x for x in arr if x not in order_map].`

Concatenate sorted buckets + remaining.

WHY elements_not_in_order at end: problem says remaining elements can be in any order.

WHY bucket sort: O(n + m) vs O(n log n) comparison sort.

Time: O(n + m). Space: O(n).

PHASE 4 CLEAN CODE

```
class _1122_RelativeSortArray:
    def relativeSortArray(self, arr1: List[int], arr2: Lis
t[int]) -> List[int]:
        freq   = Counter(arr1)
        result = []
        for num in arr2:
            result.extend([num] * freq[num])
            del freq[num]
        for num, count in sorted(freq.items()):
            result.extend([num] * count)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace relativeSortArray([2,3,1,3,2,4,3,2,4,2], order=[3,1,2])."

Count occurrences: {2:4, 3:3, 1:1, 4:2}

Build result: first add by order: 3×3=3,3,3; 1×1=1; 2×4=2,2,2,2

Then add remaining (not in order) sorted: 4,4

Result: [3,3,3,1,2,2,2,2,4,4] ok

WHY count map: O(n) to count, O(n) to rebuild in order.

Alternative: custom sort comparator using index in order array (elements not in order go last, sorted).

Elements not in order appended sorted numerically after all order-specified elements.

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(n + k log k) where k is the count of elements not in arr2 (at most n).

Overall O(n log n). Space O(n).

Given more time I'd ask: what if arr2 has elements not in arr1?

The problem guarantees that won't happen, but we'd skip them if it did.

EASY | Sorting | LeetCode #1133 | 7 of 7 | 7/7

Largest Unique Number

QUESTION

Given an integer array nums, return the largest integer that only occurs once. If no integer occurs once, return -1.

Example 1

Input	`nums = [5,7,3,9,4,9,8,3,1]`
Output	`8`
Explanation	*The maximum integer in the array is 9 but it is repeated. The number 8 occurs only once, so it is the answer.*

Example 2

Input	`nums = [9,9,8,8]`
Output	`-1`
Explanation	*There is no number that occurs only once.*

CONSTRAINTS

- 1 <= nums.length <= 2000
- 0 <= nums[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Return the largest integer that appears exactly once. If none, return -1. Right?

Values are 0–1000, array length up to 2000."

"[5,7,3,9,4,9,8,3,1]: 9 appears twice, 8 appears once -> 8 ok

[9,9,8,8]: no unique element -> -1 ok"

Can the array be empty or contain a single element? Can values be negative?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for number in numbers:
    count its occurrences by scanning
    best = max(best, with_count_1)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per number. Count all values in a table (values are at most 1000) and take the largest with count 1: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Frequency count, find max with count exactly 1 — O(n). Walk-through:"

`freq = Counter(A).`

Return max(x for x in freq if freq[x] == 1), or -1 if none.

WHY filter by count==1: 'unique' means appears exactly once.

WHY max: return the largest unique number.

Single pass: build frequency map, then single scan for max unique.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _1133_LargestUniqueNumber:
    def largestUniqueNumber(self, nums: List[int]) -> int:
        freq = Counter(nums)
        result = -1
        for num, count in freq.items():
            if count == 1 and num > result:
                result = num
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace largestUniqueNumber([5,7,3,9,4,9,8,3,1])."

Count: {5:1,7:1,3:2,9:2,4:1,8:1,1:1}

Filter unique (count==1): {5,7,4,8,1}

Return max: 8 ok

WHY simple count: O(n) pass to build count, O(n) to filter unique, O(n) to find max.

Alternative: sort and scan from right for first unique element.

Edge: no unique elements -> return -1; single element -> return it.

If no unique element exists, return -1. Linear scan after count-build: O(n) total.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

Alternative: sort descending, scan until you find a non-duplicate — O(n log n).

Counter approach is cleaner. Given more time I'd ask: what if we want the k largest

unique numbers? Collect all uniques, sort descending, take first k.